

Análise de Algoritmos Iterativos

Contagem de execuções de cada instrução I

Análise de Algoritmos Iterativos

- ▶ Comparação de **tempo de execução não é confiável**:
 - ▶ Depende do **hardware** e da **linguagem**.
 - ▶ O sistema operacional executa tarefas em **time sharing**.

Análise de Algoritmos Iterativos

A análise teórica abstrai estas características: **conta** o total de **instruções** executadas em função do **tamanho da entrada**.

- ▶ Abstrai também **diferenças** de tempo entre **instruções**.
- ▶ Se a contagem depende de valores armazenados na entrada, usamos o máximo possível (**pior caso**).
- ▶ **Tamanho** da entrada: número de **bits**.
 - ▶ Podemos simplificar para **número de elementos** na estrutura de dados, se o número de bits de cada elemento é limitado.

Análise de Algoritmos Iterativos

- ▶ **Objetivo:** Associar a cada linha do pseudocódigo uma expressão matemática que represente seu número máximo de execuções em função do tamanho da entrada.

Regras Básicas de Contagem

1. Atribuições, Retornos e Definições

Instruções fora de laços executam exatamente 1 **vez**.

2. Cabeçalho do Laço (for)

Um laço no formato `for i in range(k)` realiza:

- ▶ k iterações bem-sucedidas.
- ▶ +1 teste adicional para verificar a condição de terminação.
- ▶ Total de execuções da linha do `for`: $k + 1$.

3. Corpo do Laço

Instruções dentro de um laço simples executam k **vezes** (o número de iterações do laço).

Laços Aninhados e Somatórios

Quando temos laços aninhados, a contagem do laço interno depende do laço externo.

Usamos **somatórios** ($\sum$) para expressar essa relação:

- ▶ Se o laço externo varia o índice i de 0 até $n - 1$:

$$\sum_{i=0}^{n-1} (\text{frequência da instrução para um } i \text{ fixo})$$

- ▶ **Laço Interno** (for j in range(m)):

- ▶ Uma instrução no corpo do laço interno: $\sum_{i=0}^{n-1} \sum_{j=0}^{m-1} 1$

Exemplo Resolvido 1: Busca Linear

Analise o algoritmo abaixo que busca a ocorrência de um elemento em um vetor de tamanho n (considerando o **pior caso**, onde o elemento não é encontrado).

```
1 def busca_linear(vetor, elemento):
2     n = len(vetor)
3     posicao = -1
4     for i in range(n):
5         if vetor[i] == elemento:
6             posicao = i
7     return posicao
```

Gabarito: Exemplo 1

Linha	Instrução	Frequência (Pior Caso)
1	<code>def busca_linear...</code>	1
2	<code>n = len(vetor)</code>	1
3	<code>posicao = -1</code>	1
4	<code>for i in range(n):</code>	$n + 1$
5	<code>if vetor[i] == elemento:</code>	$\sum_{i=0}^{n-1} 1 = n$
6	<code>posicao = i</code>	0 (se nunca encontrar)
7	<code>return posicao</code>	1

Explicação

No pior caso (elemento ausente), a linha 5 é avaliada n vezes (uma para cada índice i), mas a linha 6 nunca é executada (0 vezes).

Exemplo Resolvido 2: Laços Dependentes (anotar, com as respostas)

Considere o algoritmo abaixo que opera sobre um vetor de tamanho n , onde os limites do laço interno dependem do índice i do laço externo.

```
1 def soma_triangular(A):  
2     n = len(A)  
3     total = 0  
4     for i in range(n):  
5         for j in range(i):  
6             total += A[i][j]  
7     return total
```

Gabarito: Exemplo 2

Linha	Instrução	Frequência de Execução
1	<code>def soma_triangular(A):</code>	1
2	<code>n = len(A)</code>	1
3	<code>total = 0</code>	1
4	<code>for i in range(n):</code>	$n + 1$
5	<code>for j in range(i):</code>	$\sum_{i=0}^{n-1} (1 + \sum_{j=0}^{i-1} 1) = \sum_{i=0}^{n-1} (i + 1)$
6	<code>total += A[j]</code>	$\sum_{i=0}^{n-1} i$
7	<code>return total</code>	1

Explicação da Linha 5 e 6

- ▶ Para um dado i , `range(i)` vai de 0 até $i - 1$ (i iterações bem-sucedidas +1 teste de parada = $i + 1$).
- ▶ A linha 6 executa exatamente i vezes para cada valor de i , totalizando $\sum_{i=0}^{n-1} i$.

Exercícios

Anote logo todos os enunciados.
(só precisa anotar os pseudocódigos)

Exercício: Contagem de Execuções (Pior Caso)

Considere uma lista A contendo n elementos. Analise o pseudocódigo a seguir, que verifica se existem elementos duplicados adjacentes na lista, e indique o número total de execuções de cada linha no **pior caso**.

Nota: Deixe as contagens indicadas na forma de somatórios sempre que o laço depender de índices variáveis.

```
1 def verifica_duplicados_vizinhos(A):  
2     n = len(A)  
3     contador = 0  
4     for i in range(n - 1):  
5         if A[i] == A[i + 1]:  
6             contador += 1  
7     return contador
```

Questão: Preencha a frequência de execução para as linhas 1 a 7 em função de n .

Gabarito: Contagem no Pior Caso

Linha	Instrução	Frequência de Execução
1	<code>def verifica_duplicados_vizinhos(A):</code>	1
2	<code>n = len(A)</code>	1
3	<code>contador = 0</code>	1
4	<code>for i in range(n - 1):</code>	$(n - 1) + 1 = n$
5	<code>if A[i] == A[i + 1]:</code>	$\sum_{i=0}^{n-2} 1 = n - 1$
6	<code>contador += 1</code>	$\sum_{i=0}^{n-2} 1 = n - 1$
7	<code>return contador</code>	1

Notas explicativas

- ▶ **Linha 4:** O laço itera $n - 1$ vezes com sucesso (índices de 0 até $n - 2$). A avaliação ocorre n vezes para incluir o teste de parada final.
- ▶ **Linhas 5 e 6:** No pior caso (ou caso geral de contagem sem interrupção), ambas as instruções dentro do laço são executadas para cada iteração, somando 1 para cada índice i de 0 até $n - 2$.

Exercício: Análise de Custo Linha a Linha

Considere a matriz A de dimensão $n \times n$. Analise o pseudocódigo abaixo e determine a frequência de execução de cada linha em função do tamanho da entrada n .

Nota: Expresse as frequências utilizando a notação de somatórios sempre que necessário.

```
1 def processa_matriz(A, n):  
2     soma = 0  
3     for i in range(n):  
4         for j in range(i + 1, n):  
5             soma += A[i][j]  
6             A[i][j] = soma * 2  
7     return soma
```

Questão: Preencha a tabela de frequências para as linhas 1 a 7.

Gabarito: Análise de Custo Linha a Linha

Linha	Instrução	Frequência de Execução
1	def processa_matriz...	1
2	soma = 0	1
3	for i in range(n):	$n + 1$
4	for j in range(i + 1, n):	$\sum_{i=0}^{n-1} (n - i) = \sum_{i=1}^n i$
5	soma += A[i][j]	$\sum_{i=0}^{n-1} (n - 1 - i) = \sum_{i=1}^{n-1} i$
6	A[i][j] = soma * 2	$\sum_{i=0}^{n-1} (n - 1 - i) = \sum_{i=1}^{n-1} i$
7	return soma	1

Notas explicativas

- ▶ **Linha 3:** Executa n vezes com sucesso + 1 teste final de saída do laço.
- ▶ **Linha 4:** Para cada i , o laço executa $(n - i - 1)$ vezes com sucesso + 1 teste final, resultando em $(n - i)$ avaliações.
- ▶ **Linhas 5 e 6:** Executam dentro do corpo do laço interno, exatamente $(n - 1 - i)$ vezes para cada i .

Exercício: Análise com Duas Listas

Considere duas listas, A de tamanho n e B de tamanho m . Analise o pseudocódigo abaixo, que conta quantos elementos da lista A também estão presentes na lista B .

Nota: Expresse a frequência de execução de cada linha em função de n e m utilizando a notação de somatórios sempre que necessário.

```
1 def conta_intersecao(A, B):
2     qtd = 0
3     n = len(A)
4     m = len(B)
5     for i in range(n):
6         for j in range(m):
7             if A[i] == B[j]:
8                 qtd += 1
9     return qtd
```

Questão: Determine o número de execuções de cada linha (linhas 1 a 9).

Gabarito: Análise com Duas Listas I

Linha	Instrução	Frequência de Execução
1	<code>def conta_intersecao(A, B):</code>	1
2	<code>qtd = 0</code>	1
3	<code>n = len(A)</code>	1
4	<code>m = len(B)</code>	1
5	<code>for i in range(n):</code>	$n + 1$
6	<code>for j in range(m):</code>	$\sum_{i=0}^{n-1} (m + 1) = n(m + 1)$
7	<code>if A[i] == B[j]:</code>	$\sum_{i=0}^{n-1} \sum_{j=0}^{m-1} 1 = nm$
8	<code>qtd += 1</code>	Pior caso: $\sum_{i=0}^{n-1} \sum_{j=0}^{m-1} 1$
9	<code>return qtd</code>	1

Gabarito: Análise com Duas Listas II

Notas explicativas

- ▶ **Linha 5:** Testa os n elementos da lista A mais 1 verificação de saída do laço.
- ▶ **Linha 6:** Para cada uma das n iterações do laço externo, o laço interno executa m iterações com sucesso +1 teste de saída.
- ▶ **Linha 8:** No **pior caso** (onde todos os elementos de A e B são iguais), é executada em todas as iterações do laço interno.